

docker-control — User Manual

`docker-control` is a Rust CLI that manages Docker-based PHP/LAMP projects. It gives you a single, opinionated interface for the whole lifecycle of a project: scaffolding it from a template, running its container stack locally, wiring up SSH agent forwarding and a shared reverse proxy, and cutting releases and deploying them to remote servers.

It runs as a **standalone binary**. Every command below can be invoked two equivalent ways:

Form	Example
Standalone binary	<code>docker-control start</code>
Shorthand alias (if configured)	<code>dc2 start</code>

This manual uses `docker-control` throughout; substitute whichever form you prefer.

Table of contents

1. [Installation](#)
 2. [Core concepts](#)
 3. [Quick start](#)
 4. [Global options](#)
 5. [Command reference](#)
 - [Project lifecycle](#)
 - [Working inside the stack](#)
 - [Ingress \(reverse proxy\)](#)
 - [Release & deployment](#)
 - [Custom commands](#)
 - [Maintenance & housekeeping](#)
 - [Self-management](#)
 6. [Project layout](#)
 7. [Configuration files](#)
 8. [The container stack](#)
 9. [SSH agent forwarding](#)
 10. [Deployment in depth](#)
 11. [Release & merge workflows in depth](#)
 12. [Custom control scripts](#)
 13. [Environment variables](#)
 14. [Dependencies](#)
 15. [Troubleshooting](#)
-

1. Installation

`docker-control` is distributed via Homebrew.

If you don't have Homebrew, install it first:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Trust the tap. Modern Homebrew (with `HOMEbrew_REQUIRE_TAP_TRUST` set) refuses to load formulae from a non-official tap until you explicitly trust it:

```
brew trust INTERLIGENT-kommunizieren-GmbH/tap
```

Then install the tool (`brew install` taps the repository automatically):

```
brew install INTERLIGENT-kommunizieren-GmbH/tap/docker-control
```

This makes the `docker-control` command available.

Note: if `brew install` fails with a trust error, run the `brew trust` command above first. Trusted entries are stored in `~/.homebrew/trust.json` (or under `$XDG_CONFIG_HOME/homebrew/`).

Building from source

The project is Rust 2024 edition. To build and test locally:

```
cargo build --release      # release binary in target/release/docker-control
cargo nextest run          # run the test suite (use nextest, not `cargo test`)
```

Installing dependencies

`docker-control` shells out to a number of external tools (see [Dependencies](#)). To install every Homebrew-installable dependency in one step — including optional ones:

```
docker-control install-deps
```

On startup, the tool also checks for missing dependencies and, on macOS / native Linux with Homebrew, offers to install the ones it can.

2. Core concepts

Managed project. Most commands require the current directory to be a *managed project*, marked by a `.managed-by-docker-control` sentinel file. `docker-control init` creates this file. Commands that mutate a project (`start`, `stop`, `build`, `console`, `setacl`, `update`, ...) refuse to run outside a managed directory.

Two repositories. A project directory is the **infrastructure / operations** layer (the compose stack, config, secrets). The actual PHP **application lives in `htdocs/`**, which is a **separate git repository** mounted into the `php` container at `/var/www/html`. Release, merge, and deploy operate on the `htdocs/` repo (and optionally vendor modules under `htdocs/vendor/<name>/`).

Template-driven. New projects are scaffolded from an embedded template. The template is compiled into the binary and extracted to your OS config directory on first run (and whenever the version changes). `docker-control update` re-syncs an existing project with the current template.

Ingress. An optional shared reverse-proxy stack that routes `https:// dev` URLs to your project containers. It is a separate compose stack from your project.

3. Quick start

```
# 1. Create and enter a project directory
mkdir my-app && cd my-app

# 2. Scaffold the project (interactive: name, PHP version, optional git clone)
docker-control init

# 3. (Optional, once per machine) start the shared reverse proxy
docker-control start-ingress

# 4. Start the container stack
docker-control start

# 5. Trust the ingress CA (once per host) – only works after a project has been
#    started, since the CA is generated when the first domain is signed
docker-control trust-ca

# 6. Open a shell in the PHP container to run composer, bin/console, etc.
docker-control console

# 6. Check status at any time
docker-control status
```

Your app is then reachable at `https://<name>.lvh.me/` (the `lvh.me` domain resolves to `127.0.0.1`).
phpMyAdmin is at `/_phpmyadmin/` and Mailpit at `/_mail/`.

4. Global options

These flags apply to (nearly) every command:

Flag	Description
<code>-d, --dir <DIRECTORY></code>	Operate on the given project directory instead of the current one.
<code>--debug</code>	Enable verbose debug output.
<code>-h, --help</code>	Show help. With no subcommand, help also prints a full project status summary and lists any custom commands.
<code>-V, --version</code>	Print the <code>docker-control</code> version.

SSH-agent daemon control flags (handled before normal command parsing):

Flag	Description
<code>--start-ssh-agent</code>	Start the SSH agent forwarding daemon.
<code>--stop-ssh-agent</code>	Stop the SSH agent forwarding daemon.
<code>--restart-ssh-agent</code>	Restart the SSH agent forwarding daemon.

See [SSH agent forwarding](#) — normally you don't need these because the daemon is started automatically.

Running `docker-control` with **no command** prints the project status and a hint to run `--help`.

5. Command reference

Project lifecycle

init

Initialize the current (or `--dir`) directory as a managed project from the template.

Interactively prompts for:

- **Project name** (defaults to the directory name; sanitized to lowercase with separators replaced by `_`).
- **PHP version** — one of `7.4`, `7.4-oci`, `8.2`, `8.2-oci`, `8.5`, `8.5-oci`.
- Optionally **clone a git repository into `htdocs/`** (SSH URL recommended).

It copies the template, renames `.gitignore-dist` → `.gitignore`, creates the `htdocs/` directory, applies host ACLs, auto-selects a free database host port in the range `33060-33099`, and writes a `.env` file. If the directory is non-empty (and not already managed) it warns and asks for confirmation before overwriting.

```
docker-control init
```

start

Start the project containers in detached mode (`docker compose up -d`). Before starting, it does a throttled check (at most weekly) for outdated container images and offers to pull them. After starting, it re-applies host and container ACLs on `htdocs`.

```
docker-control start
```

stop

Stop the project containers (`docker compose down`).

```
docker-control stop
```

restart

Stop then start the containers. Also runs the outdated-image check and re-applies ACLs.

```
docker-control restart
```

status

Show a detailed project status report — plugin management, git repository state (branch + uncommitted changes), deployment config and configured environments, and Docker container status — followed by `docker compose ps`.

```
docker-control status
```

build [args...]

Build the project containers. Any extra arguments are passed straight through to `docker compose build`.

```
docker-control build
docker-control build --no-cache
```

pull

Pull the latest images for the project (`docker compose pull`).

```
docker-control pull
```

Working inside the stack

console [container]

Open an interactive `bash` shell inside a container. Defaults to the `php` container, entered as the `www-data` user with the working directory at `/var/www/html` . Pass a service name to target a different container; pass `help` to list available services.

```
docker-control console      # php container as www-data
docker-control console db   # db container
docker-control console help # list services
```

Run `composer` , `php` , `bin/console` , `artisan` , etc. **inside** the `php` container via `console` , since the application is mounted there. MariaDB is also reachable from the host at `127.0.0.1:${DB_HOST_PORT}` .

setacl

Re-apply host and container ACL permissions on `htdocs` without restarting. Use this when host/container file permissions drift out of sync. The project containers must be running.

```
docker-control setacl
```

Ingress (reverse proxy)

The ingress is a shared reverse-proxy compose stack that serves your projects over HTTPS. You typically start it once per machine.

Command	Effect
<code>start-ingress</code>	Start the ingress containers (detached). Also syncs ingress volumes.
<code>stop-ingress</code>	Stop the ingress containers.
<code>restart-ingress</code>	Stop then start the ingress containers.
<code>status-ingress</code>	Show ingress container status.
<code>pull-ingress</code>	Pull the latest ingress images.
<code>trust-ca</code>	Trust the ingress CA certificate on this host.

trust-ca

Installs the ingress proxy's self-signed CA certificate into the host trust store so your project's `https://dev` URLs are trusted without browser warnings. It handles:

- **macOS** — the System keychain (via `sudo security add-trusted-cert`).
- **Linux / WSL** — the system anchor store (Debian/Ubuntu or RHEL layout).
- **Windows** — the Windows certificate store.
- **Browsers** — the NSS trust store used by Chrome/Chromium and Firefox (via `certutil` , from the Homebrew `nss` formula) on macOS/Linux.

The CA does not exist until the first domain is signed. The self-sign companion only generates the CA when it signs a project's first domain, so `start-ingress` alone is not enough — you must start at least one project first. If no CA is found, `trust-ca` tells you to start the ingress/a project first. Once a project is up, run `trust-ca` once per host. It may prompt for `sudo /admin` rights. Restart your browser afterward.

```
docker-control start-ingress # start the shared proxy
docker-control start        # start a project (signs the first domain → CA is created)
docker-control trust-ca     # trust the now-existing CA
```

Release & deployment

add-deploy-config

Interactively add a deployment environment to `.deploy.json` (creating the file if needed). Prompts for environment name, SSH user, domain, server root, console command, description, optional Microsoft Teams webhook, optional COPS integration, and shared directories/files. See [Configuration files](#).

```
docker-control add-deploy-config
```

Work in progress. The `release` and `deploy` commands are still under active development and their behavior may change. When migrating a project, **Capistrano is currently used** for deployment rather than `docker-control deploy`.

release [module]

Create a new release with automatic semantic versioning. Optionally target a vendor module (otherwise you're prompted to choose the main project or a vendor module when modules exist). See [Release & merge workflows in depth](#).

```
docker-control release
docker-control release some-vendor-module
```

merge [module]

Cherry-pick a release branch back into the primary branch via an isolated `<release>-merge` branch, skipping `release: -`prefixed commits, then push for a PR. See [Release & merge workflows in depth](#).

```
docker-control merge
```

deploy <env> [options]

Deploy a selected release/tag to a configured environment. See [Deployment in depth](#).

Option	Description
<code><env></code>	Target environment name (must exist in <code>.deploy.json</code>). Required.
<code>-r, --release <REL></code>	Deploy a specific release/tag, skipping interactive selection.
<code>--maintenance-mode <hard soft></code>	Maintenance mode to use with <code>--yes</code> (default <code>hard</code>).
<code>-y, --yes</code>	Skip all interactive prompts (non-interactive deploy).

```
docker-control deploy production
docker-control deploy staging --release 2.4.1
docker-control deploy production --yes --maintenance-mode soft
```

Custom commands

create-control-script <name>

Scaffold a new custom control script. Prompts for a description, whether the script should override a built-in command of the same name, and where to store it (inside `htdocs` or in the project root). See [Custom control scripts](#).

```
docker-control create-control-script my-command
```

<script-name> [args...]

Any shell script under `control-scripts/` or `htdocs/.docker-control/control-scripts/` is dispatched as a subcommand. Custom commands appear in `--help` with their descriptions.

```
docker-control my-command arg1 arg2
```

Maintenance & housekeeping

show-running

List all running projects managed by `docker-control` across the machine (project name and directory), discovered via Docker container labels.

```
docker-control show-running
```

update [--yes]

Re-sync **this project** with the current template. It stops the project (if running), creates a timestamped `backup_<epoch>/` of the current project files, copies the template over the project (excluding `logs/` and `volumes/`), merges `.gitignore-dist` into `.gitignore`, and restarts the project if it had been running.

This **rewrites project files**, so it asks for confirmation. In a non-interactive context it refuses unless `--yes` is given.

`update` updates the *project*. To upgrade the *tool itself*, use `upgrade`.

```
docker-control update
docker-control update --yes      # non-interactive
```

cleanup-backups [options]

Remove old `backup_*` folders left behind by `update` / `migrate` (and clean up their PhpStorm exclude entries).

Option	Description
<code>-k, --keep <N></code>	Keep the N most recent backups (default 5). Deletes the rest.
<code>--older-than <DAYS></code>	Delete backups older than DAYS days. Mutually exclusive with <code>--keep</code> .
<code>--dry-run</code>	List what would be deleted without deleting.

Option	Description
<code>-y, --yes</code>	Skip the confirmation prompt.

```
docker-control cleanup-backups --dry-run
docker-control cleanup-backups --keep 3
docker-control cleanup-backups --older-than 30 --yes
```

Self-management

`install-deps`

Install every Homebrew-installable dependency (critical and optional) in one `brew install`. This command intentionally skips the normal dependency check so it can be run to fix missing dependencies. Requires Homebrew.

```
docker-control install-deps
```

`upgrade`

Upgrade `docker-control` itself via Homebrew (`brew upgrade .../docker-control`). The tool also performs a throttled (weekly) background check and, in an interactive terminal, may offer to upgrade when a newer version is available.

```
docker-control upgrade
```

6. Project layout

A scaffolded project looks like this:

Path	Purpose
<code>compose.yml</code>	The service stack definition. Template-owned.
<code>htdocs/</code>	The application — a separate git repo (gitignored here). Mounted at <code>/var/www/html</code> .
<code>htdocs/.docker-control/</code>	Per-project config: <code>.deploy.json</code> , <code>control-scripts/</code> , <code>deployment-scripts/<env>.rhai</code> .
<code>config/</code>	Container config: <code>apache-sites/</code> , <code>php.ini</code> , <code>mariadb.cnf</code> , <code>composer.config.json</code> , <code>crontab</code> , <code>htpasswd</code> , <code>ssmtp.conf</code> .
<code>secrets/</code>	DB credential files: <code>db_name.txt</code> , <code>db_user.txt</code> , <code>db_pw.txt</code> , <code>db_root_pw.txt</code> .
<code>volumes/</code>	Persistent data: <code>db/</code> , <code>valkey/</code> , <code>composer-cache/</code> , <code>db-share/</code> .
<code>logs/</code>	Container logs: <code>apache/</code> , <code>mariadb/</code> .
<code>releases/</code>	Git worktrees used during <code>release</code> / <code>merge</code> .
<code>deployments/</code>	Deployment archives and optional Rhai hook scripts.
<code>control-scripts/</code>	Project-local custom subcommands.
<code>.env</code>	Project settings (see below).
<code>.managed-by-docker-control</code>	Sentinel marking the directory as managed.

7. Configuration files

`.env`

Written by `init` and read at runtime (and by `compose.yml`).

Key	Meaning
<code>PROJECTNAME</code>	Project name; the compose project name and container label.
<code>BASE_DOMAIN</code>	Dev domain, e.g. <code>myapp.lvh.me</code> (resolves to <code>127.0.0.1</code>).
<code>ENVIRONMENT</code>	App environment, e.g. <code>development</code> (maps to <code>APP_ENV</code>).
<code>DB_HOST_PORT</code>	Host port mapped to MariaDB <code>3306</code> (auto-selected at init).
<code>PHP_VERSION</code>	Tag for the <code>fduarte42/docker-php:<PHP_VERSION></code> image.
<code>XDEBUG_IP</code>	Host IP Xdebug connects back to (default <code>host.docker.internal</code>).
<code>IDE_KEY</code>	Xdebug IDE key.

Optional overrides consumed by `compose.yml` include `PHP_GC_MAX_LIFETIME`, `PHP_UPLOAD_LIMIT`, and `PHP_ERROR_REPORTING`.

`.deploy.json`

Deployment configuration. Search order:

1. `htdocs/.docker-control/.deploy.json` (preferred)
2. `.deploy.json` (project root, fallback)

The `version` field must be `"1.0"`. Example:

```
{
  "version": "1.0",
  "environments": {
    "production": {
      "user": "deploy",
      "domain": "production.example.com",
      "serviceRoot": "/var/www/html",
      "console_command": "bin/console",
      "description": "Production environment - stable releases only",
      "tags": ["production", "critical"],
      "teamsWebhookUrl": "https://outlook.office.com/webhook/...",
      "copsIntegration": false,
      "sharedDirectories": ["var/log", "public/uploads"],
      "sharedFiles": [".env.local"]
    },
    "staging": {
      "user": "deploy",
      "domain": "staging.projects.interlignent.com",
      "serviceRoot": "/var/www/html",
      "description": "Staging environment for testing"
    }
  },
  "environmentOrder": ["production", "staging"],
  "defaults": {
    "serviceRoot": "/var/www/html",
    "domainSuffix": ".projects.interlignent.com"
  }
}
```

Per-environment fields:

Field	Required	Meaning
<code>user</code>	yes	SSH user on the target server.
<code>domain</code>	yes	Target server hostname.
<code>serviceRoot</code>	no	Deploy root on the server (default <code>/var/www/html</code>).
<code>console_command</code>	no	Console entry point (default <code>bin/console</code>).
<code>description</code>	no	Human-readable description.
<code>tags</code>	no	Free-form labels.
<code>branch</code>	no	Associated branch (informational).
<code>teamsWebhookUrl</code>	no	Microsoft Teams webhook for deploy notifications.
<code>copsIntegration</code>	no	Run COPS <code>cops:outdated</code> / <code>cops:permissions</code> during deploy.
<code>sharedDirectories</code>	no	Directories symlinked from <code><serviceRoot>/shared/</code> into each release.
<code>sharedFiles</code>	no	Files symlinked from <code><serviceRoot>/shared/</code> into each release.

The easiest way to create/extend this file is `docker-control add-deploy-config`.

8. The container stack

`compose.yml` defines the following services:

Service	Image	Role
php	fduarte42/docker-php:\${PHP_VERSION}-debug	Apache + mod_php serving <code>htdocs/</code> at <code>/var/www/html</code> .
db	mariadb:11.8	Database; host port <code>\${DB_HOST_PORT}</code> → <code>3306</code> .
phpmyadmin	phpmyadmin	DB admin UI at <code>/_phpmyadmin/</code> .
mail	axllent/mailpit	Catch-all mail UI at <code>/_mail/</code> .
gotenberg	gotenberg/gotenberg:8-chromium	PDF/document conversion service.
redis	valkey/valkey:8.1	Redis-compatible cache/store.
logrotate	ghcr.io/fduarte42/logrotate	Rotates container logs.

Networking: the `frontend-tier` is the external `proxy` network (the ingress); the `backend-tier` is an internal bridge. DB credentials are provided via Docker `secrets` sourced from `secrets/*.txt`.

Prefer `docker-control` commands over raw `docker` / `docker compose`, because the CLI also wires up SSH agent forwarding, ACL fixes, ingress, secrets, and env handling.

9. SSH agent forwarding

Deploy and release run `composer install` inside Docker containers that need access to your SSH keys (e.g. to fetch private Composer packages). To make the host SSH agent available to containers, `docker-control` runs a small **forwarding daemon** that exposes the agent on TCP port `2222` (`SSH_AGENT_PORT`).

- The daemon is **started automatically** when missing (unless `DOCKER_CONTROL_SKIP_SSH_AGENT` is set, or the command doesn't need SSH). Once running, the tool publishes `SSH_AUTH_PORT=<bind_ip>:2222` for compose and deploy commands.
- The bind IP is platform-dependent (`127.0.0.1` on macOS/WSL/Docker Desktop; the `docker0` bridge IP on native Linux).
- The daemon PID/logs live at `/tmp/docker-control-ssh-agent.{pid,log,err}`.

Manual control:

```
docker-control --start-ssh-agent
docker-control --stop-ssh-agent
docker-control --restart-ssh-agent
```

If you see “SSH agent forwarding is not available”, ensure `ssh-agent` is running and `SSH_AUTH_SOCK` is set in your shell.

10. Deployment in depth

Work in progress. `deploy` is under active development and may change. When migrating a project, **Capistrano is currently used** for deployment instead.

`docker-control deploy <env>` runs, in order:

1. **Load config** for `<env>` from `.deploy.json`.
2. **Select a release** — fetches tags and prompts (or uses `--release`).
3. **Confirm** the deployment (skipped with `--yes`).
4. **Teams "started" notification** (if `teamsWebhookUrl` is configured).
5. **Build the archive** locally: check out the release tree with git, run `composer install -o` inside a `fduarte42/docker-php:<PHP_VERSION>` container (using SSH agent forwarding), then compress with `7z` into `deployments/<timestamp>_<release>.7z`.
6. **Transfer & extract** the archive to `<serviceRoot>/releases/<timestamp>_<release>/` over `scp` / `ssh`, then remove the archive.
7. **Prune** old releases, keeping the 5 most recent (never the live `current`).
8. **Shared paths** — symlink each `sharedDirectories` / `sharedFiles` entry from `<serviceRoot>/shared/` into the new release.
9. **Reload FPM** (`sudo php-fpm-reload.sh`).
10. **Maintenance mode** — enable `hard` or `soft` on the current and new releases.
11. **Hooks: `pre_deploy`** (Rhai; see below).
12. **Deployment tasks** — clear opcode, clear ORM metadata/query caches; optionally clear result cache, run `migrations:migrate`, and run `orm:schema-tool:update` (each prompted unless `--yes`).
13. **COPS integration** (if enabled) — `cops:outdated` and `cops:permissions`, with prompts to continue on failure.
14. **Hooks: `post_deploy`**.
15. **Manual pause** — in interactive mode, waits for ENTER so you can run ad-hoc commands on the server before going live.
16. **Flip the `current` symlink** to the new release.
17. **Disable maintenance mode** and do a final opcode clear.
18. **Hooks: `done_deploy`**.
19. **Teams "success"/"failed" notification.**

On failure the local archive is removed and (if configured) a "failed" Teams notification is sent; maintenance mode is best-effort restored.

Deployment hooks (Rhai)

You can inject custom SSH steps at three points: `pre_deploy`, `post_deploy`, and `done_deploy`. Hooks are [Rhai](#) scripts loaded from the first of:

1. `htdocs/.docker-control/deployment-scripts/<env>.rhai`
2. `deployments/scripts/<env>.rhai`

Each hook is an optional function receiving `(console_current, release_dir, console_new, server_root)` and can call `exec_ssh(command)` to run commands on the target server:

```
fn pre_deploy(console_current, release_dir, console_new, server_root) {  
    exec_ssh(console_new + " app:warmup");  
}  
  
fn post_deploy(console_current, release_dir, console_new, server_root) {  
    exec_ssh("echo Deployed " + release_dir);  
}
```

Functions you don't define are simply skipped.

11. Release & merge workflows in depth

Work in progress. The `release` workflow is still under active development and may change.

Both workflows operate on the `htdocs/` repository (or a vendor module under `htdocs/vendor/<name>/`) using isolated git **worktrees** under `releases/` so your working tree is never disturbed.

release

1. Optionally select the main project or a vendor module.
2. Pre-flight: `fetch`, determine the primary branch, update it.
3. Determine the version:
 - No release branches yet → initial release `1.0.x`.
 - **Breaking change?** → next major `X.0.x`.
 - **New feature?** → next minor `X.Y.x`.
 - Otherwise → **patch**: pick an existing `X.Y.x` branch, next patch `X.Y.Z`.
4. **Minor/major/initial** create a new `X.Y.x` **branch**: update `composer.json` version to `X.Y.x-dev`, generate `composer.lock` via Docker, commit both, generate a `CHANGELOG.md`, and push the branch.
5. **Patch** creates a **tag** on the `X.Y.x` branch: update `composer.json` to the exact version, generate the changelog, create tag `X.Y.Z`, and push the tag.

`release:` -prefixed commits are used for the `composer.json/lock` bookkeeping so they can be excluded when merging back.

merge

1. Optionally select the main project or a vendor module.
2. Pre-flight: `fetch`, determine primary branch, list release branches, pick one.
3. Create two worktrees: the release branch, and a new `<release>-merge` branch based on `origin/<primary>`.
4. Compute the commits on the release branch not in primary, **excluding** `release: commits`, and confirm.
5. Cherry-pick each commit. On conflict, choose to **launch the merge tool**, mark it **resolved**, or **abort**.
6. On success, push `<release>-merge` and print next steps for opening a PR from `<release>-merge` into the primary branch. The local merge branch is cleaned up on a successful push; on failure or if you decline the push, it's preserved for inspection.

12. Custom control scripts

Custom commands are plain `bash` scripts discovered from (in priority order):

1. `htdocs/.docker-control/control-scripts/`
2. `control-scripts/`

Create one with `docker-control create-control-script <name>`. It's run with the project directory as the working directory and `PROJECT_DIR` set in its environment.

A script supports two special probe arguments:

- `_desc_` — print a one-line description (shown in `--help`).
- `_override_` — print `true` to always win a name clash with a built-in command.

Generated skeleton:

```
#!/bin/bash
set -e

if [[ "$1" == "_desc_" ]]; then
    echo "My command description"
    exit 0
fi

# Optional: always override a same-named built-in command
if [[ "$1" == "_override_" ]]; then
    echo "true"
    exit 0
fi

echo "my-command - WAITING FOR IMPLEMENTATION"
exit 0
```

Name clashes

If a custom script has the same name as a built-in command (e.g. `build.sh`), you'll be prompted to choose which runs. Pressing Enter/Escape keeps the built-in. Add the `_override_` block to make your script always win without a prompt. When a custom script wins a clash with a built-in that requires a managed project, the managed-project check is still enforced.

13. Environment variables

Variable	Effect
<code>DOCKER_CONTROL_SKIP_DEPENDENCY_CHECK</code>	Skip external tool dependency checks.
<code>DOCKER_CONTROL_SKIP_SSH_AGENT</code>	Skip SSH agent management.
<code>DOCKER_CONTROL_SKIP_IMAGE_CHECK</code>	Skip the outdated-image check on start/restart.
<code>DOCKER_CONTROL_SKIP_SELF_UPDATE_CHECK</code>	Skip the weekly self-update check.
<code>DOCKER_CONTROL_INGRESS_DIR</code>	Override the ingress directory location.
<code>DOCKER_CONTROL_TEMPLATE_DIR</code>	Override the template directory location.
<code>HOMEbrew_PREFIX</code>	Override the detected Homebrew prefix (used for ingress paths).
<code>SSH_AUTH_PORT</code>	Set automatically to <code><bind_ip>:2222</code> ; read by <code>deploy/release</code> Docker commands.
<code>SSH_AUTH_SOCKET</code>	Standard SSH agent socket; required for agent forwarding.
<code>PHP_VERSION</code>	Read from the project <code>.env</code> ; used for the <code>fduarte42/docker-php</code> image tag.

14. Dependencies

External tools `docker-control` relies on. Critical tools must be present or commands abort; optional tools only enable certain features. On macOS / native Linux with Homebrew, missing tools that have a formula can be installed interactively or via `install-deps`.

Tool	Critical	Used for
Docker (≥ 20.10)	yes	All container operations.
Docker Compose (≥ 2.4)	yes	Managing the service stack.
Git	yes	Release and merge workflows.
SSH	yes	Remote access during deploy.
SCP	yes	File transfers during deploy.
Bash	yes	Executing custom scripts.
Sudo	no	Elevated privileges (migration, <code>trust-ca</code>).
Rsync	no	Migration tasks.
7-Zip (<code>7z</code> , <code>sevenzip</code> formula)	no	Building deployment packages.
setfacl / getfacl	no	Host/container ACLs on Linux (macOS falls back to <code>chmod</code>).
certutil (<code>nss</code> formula)	no	Browser CA trust for <code>trust-ca</code> .

Docker/Docker Compose/SSH/SCP are not installed by `install-deps` (Docker needs a daemon; Homebrew's `openssh` is keg-only), so install those yourself.

15. Troubleshooting

"... not managed by docker control plugin". The directory lacks `.managed-by-docker-control`. Run `docker-control init`, or use `--dir` to point at the right project.

"No deployment configuration found (.deploy.json)". Run `docker-control add-deploy-config` to create one.

"No tags found in repository" on deploy. Cut a release first with `docker-control release`.

Composer can't reach private repos during release/deploy. The SSH agent forwarding daemon may not be running. Check that `ssh-agent` is up and `SSH_AUTH_SOCK` is set, then `docker-control --restart-ssh-agent`.

trust-ca reports no CA certificate found. The CA is only generated when the self-sign companion signs the first domain. Start the ingress **and** at least one project (`docker-control start-ingress` then `docker-control start`) before running `trust-ca`.

Browser still warns about the dev HTTPS cert. Ensure the ingress and a project are running so the CA exists, run `docker-control trust-ca`, then restart the browser. On Linux, `certutil` (Homebrew `nss`) is needed for browser trust.

setacl says containers aren't running. Start the project first with `docker-control start`.

update refuses to run. In a non-interactive shell it won't rewrite a project without `--yes`. Remember `update` rewrites the *project* from the template (a backup is created); to upgrade the *tool*, use `upgrade`.

Too many backup_* folders after updates. Clean them with `docker-control cleanup-backups` (`--dry-run` first to preview).

Docker image version warnings on start. `docker-control` checks image freshness weekly and offers to pull. Set `DOCKER_CONTROL_SKIP_IMAGE_CHECK=1` to disable.

This manual documents `docker-control` and is kept alongside the source. For a concise command list, run `docker-control --help`.